Task 4: speeding up the block scan

1. What is the problem?

An *inclusive scan* replaces each element with the sum of all elements up to and including it: $[2, 1, 3] \to [2, 3, 6]$. The GPU divides the array into pieces, called *tiles*, and assigns each piece to a block. Each block scans its own piece, but must also add the sum of all preceding pieces to each result. This sum is called the **offset**.

Task 4 speeds up the search for the offset, called the *look-back*, while keeping the scan to a single pass over the array: one input read and one output write per element, plus accesses to coordination data.

2. How does the baseline find the offset?

Each block publishes a descriptor, consisting of a state and its associated value, visible to other blocks:

- **X**: the block has not yet published a result;
- **A**: the *aggregate* is available: the sum of the elements in this block alone;
- **P**: the *prefix* is available: the sum of all blocks up to and including this one.

Block $k > 0$ first publishes its own aggregate A. A thread then reads descriptors backwards from $k - 1$: on **A**, it adds the aggregate and continues; on **X**, it waits and reads again; on **P**, it adds the prefix and stops, because that prefix already includes the earlier blocks. It then publishes its own P, equal to the offset plus its aggregate, and applies the offset to the local scan. Block 0 has a zero offset and publishes P directly.

Example. Block 5 reads, in order, $A_4 = 4$, $A_3 = 7$ and $P_2 = 10$. Its offset is $4 + 7 + 10 = 21$: its local values $[2, 3]$, scanned as $[2, 5]$, become $[23, 26]$. The block then publishes $P_5 = 26$. An X encountered before P_2 cannot be treated as zero: its value is not yet known. For other associative operators, values are prepended to the offset to preserve the order of the blocks.

3. What do the optimizations change?

- **4.1 – Warp-wide reads.** A warp is a group of 32 threads. Instead of reading predecessors one at a time, the warp reads them in groups of 32. One thread examines the results; if it encounters X before P, the group is read again. This reduces the wait for successive memory accesses.
- **4.2 – Parallel look-back.** The same warp finds P and X and combines values in parallel, rather than leaving the work to one thread. It follows the same rule for X; if it finds neither P nor X, it moves to the previous 32 descriptors.
- **4.3a – Packed descriptors.** For values up to 32 bits, the state and value are read together in one atomic word, avoiding two dependent reads. For 64-bit values, two words carry a copy of the state; if the states do not match, the read is retried.
- **4.3b/c – Other experiments.** *Skip-and-revisit* temporarily skips X entries, then walks forward over the interval and waits for any that are still X. If it encounters a new P, it replaces the running total because that prefix already includes the earlier blocks. Debug counters measure waits and the distance to the P found.
- **4.4–4.6 – Local work and windows.** The experiments include a local scan with fewer memory conflicts, a window of 128 descriptors, and reuse of previously read values, re-reading only the X entries that block progress. A published aggregate A remains valid even if its block advances to P.

4. How is correctness ensured and measured?

Blocks receive a dynamic number using `atomicAdd` and use it to choose their piece. A block therefore waits only for blocks that have already started, without depending on the order chosen by the GPU. With separate descriptors, the writer publishes the value first and then the state with *release* semantics; the reader uses *acquire* on the state before reading the value. Thus, when it sees A or P, it reads the corresponding value. States and the counter are reset before every launch.

`make test_lookback` runs the stress tests; `make bench` compares each variant with a CPU scan before timing it: the comparison is exact for integers and uses a tolerance for floating-point values. Effective bandwidth is $2n\,\mathrm{sizeof}(T)/t$, with n elements of type T and time t; it is compared with a GPU `memcpy` and the scan from NVIDIA's CUB library. On an A100, for 32-bit integers and 256 MiB, the baseline reaches 53.2% of `memcpy`; the new local scan, parallel warp and packed descriptors together reach 91.4% (CUB: 93.9%). These measurements come from a single session on a shared GPU.

The code is in `pbb_kernels.cuh`; tests and benchmarks are in `bench_lookback.cu`.